

Credit Name: File Structures & Exception Handling
Assignment Name: WordGuess.java

The following questions can help you in thinking critically about your problem-solving processes:

Understanding the Problem:

How did you approach understanding the challenge?

I approached the challenge by first thinking about the processes and how it would work before I start coding. Something that helped a lot is seeing the template so I have a visual idea and mental note of what I could do to make the code and for it to actually work.

```
/* select secret word */
try {
    //initialize the file reader object to name of the file object

    //initialize the BufferedReader object to the name of the file reader as a parameter

    //Get the number of words in the file using readfile
    //update the word to guess to the random object and number of words read plus one

    //iterate through the word to guess slots
    //update the secret work from the lines read from the file

    //close your BufferedReader object
    //close your FileReader object
} catch (/* ?? */)
{
    /* ?? */
} catch (/* ?? */)
{
    /* ?? */
}
```

Figuring out how to make this part work with no error is usually the hardest because this is where the file reads and inputs everything

Were there any parts of the problem you found confusing at first? If so, how did you resolve that confusion?

To resolve the problem I would ask a friend or chatgpt because that was the most efficient way to get my questions answered without waiting. One of the parts that I found confusing was making the code pick a word from the files. I fixed that by asking chatgpt because it told me a quick solution and showed me taught me how to do it so I wouldn't need its help

```
try { // starts file reading section

    dataFile = new File("../Chapter11/src/Mastery/words.txt"); // opens the words file

    in = new FileReader(dataFile); // connects file reader to the file
    readFile = new BufferedReader(in); // connects buffered reader

    numWords = Integer.parseInt(readFile.readLine()); // reads number of words from first line

    wordToGuess = (int)(Math.random() * numWords) + 1; // picks random word number
```

This part took me a while to figure out because I didn't really know how to make the program pick a word from the file so the user can guess it and chatgpt told me how and gave a code that will work in my level of computer science.

Planning the Solution

Did you create a plan or break the problem into smaller steps before coding?

I broke the code into smaller steps before coding because that would be easier to catch errors and be more efficient to think about what to do instead of pondering on the whole code and

```
final String FLAG = "!"; // special symbol used to quit the game
FileReader in; // reads the file
BufferedReader readFile; // reads full lines from the file
File dataFile; // stores the word file
Scanner input = new Scanner(System.in); // gets user input

int numWords, wordToGuess; // stores total words and random word number
String secretWord = ""; // stores the chosen word
String wordSoFar = ""; // stores guessed progress using dashes
String updatedWord = ""; // stores updated guessed letters
String letterGuess; // stores the guessed letter
int numGuesses = 0; // counts guesses

try { // starts file reading section

    dataFile = new File("../Chapter11/src/Mastery/words.txt"); // opens the words file

    in = new FileReader(dataFile); // connects file reader to the file
    readFile = new BufferedReader(in); // connects buffered reader

    numWords = Integer.parseInt(readFile.readLine()); // reads number of words from first line

    wordToGuess = (int)(Math.random() * numWords) + 1; // picks random word number

    for(int i = 0; i < wordToGuess; i++) { // loops until reaching chosen word
        secretWord = readFile.readLine(); // stores chosen word
    }
}
```

trying to figure it out step by step.

First I made the variables and then I added the variables and after made the try statement and then programmed the code to pick a word and count number of guesses

How did you decide on the tools, data structures, or algorithms to use?

I wrote the code in small pieces instead of trying to do the whole thing at once. First, I focused on the file-reading part to make sure the program could open words.txt, read the number of words, and correctly choose a random word. After that was working, I built the display section that creates the dashes for the hidden word. Next, I added the guessing loop, which asks the user for a letter and updates the word step by step. Finally, I tested the ending conditions for winning or quitting and added the guess counter. Breaking it into smaller parts made it much easier to find mistakes and make sure each section worked before moving on.

```
    }
    while (!letterGuess.equals(FLAG) && !wordSoFar.equals(secretWord)); // repeats until quit or solved

    if(wordSoFar.equals(secretWord)) { // checks if player won
        System.out.println("Congratulations! You guessed the word: " + secretWord); // winning message
    }
    else {
        System.out.println("Game ended. The word was: " + secretWord); // losing message
    }

    System.out.println("Number of guesses: " + numGuesses); // shows guess count
}
```

This is the guessing loop to determine the amount of guesses and if they wanted to quit

Implementation

Did you write the code in small pieces or attempt the entire solution at once?

For the implementation, I wrote the code in small pieces instead of attempting the whole solution at once. I first made sure the file-reading section worked so the program could successfully open the word file and pick a random word. Once that was working, I moved on to creating the hidden word display with dashes. After that, I built the guessing loop, which updates the word each time the user enters a letter. Last, I added the ending conditions for winning or quitting along with the guess counter.

```
for(int i = 0; i < secretWord.length(); i++) { // creates dashes equal to word length
    wordSoFar = wordSoFar + "-"; // adds one dash
}
```

This is the dashes used for the the words to be displayed the user can know how many letter the secret word is

How did you test your solution along the way to make sure it was working?

I tested my solution step by step as I built it. First, I checked that the program could read the file and choose a random word correctly. Then I tested whether the dashes matched the word length. After that, I tried different letter guesses to make sure the word updated properly and repeated letters were shown. Finally, I tested both ending conditions, solving the word and quitting with !, to make sure the game ended correctly. This helped me catch problems early and fix them before moving on.

```
Submitter: WordGuess.java AppMain
WordGuess game.
-----
Enter a letter (! to quit):
```

Overcoming Challenges

What part of the problem was the most difficult for you?

The most difficult part of the problem was the try statement for reading the file. It was challenging to understand how the program uses try and catch to safely open words.txt, read the lines, and handle errors if the file path is wrong or the file is missing. At first, it was confusing to connect the File, FileReader, and BufferedReader together in the correct order. Once I understood that the try block runs the file-reading code and the catch block prevents the program from crashing, the rest of the program made much more sense.

```
try { // starts file reading section

    dataFile = new File("../Chapter11/src/Mastery/words.txt"); // opens the words file

    in = new FileReader(dataFile); // connects file reader to the file
    readFile = new BufferedReader(in); // connects buffered reader

    numWords = Integer.parseInt(readFile.readLine()); // reads number of words from first line

    wordToGuess = (int)(Math.random() * numWords) + 1; // picks random word number

    for(int i = 0; i < wordToGuess; i++) { // loops until reaching chosen word
        secretWord = readFile.readLine(); // stores chosen word
    }
```

How did you handle moments when you felt stuck or unsure of what to do next?

When I felt stuck, I went back to the try statement and file-reading example from the skill builders and past lessons. Looking at how the File, FileReader, and BufferedReader were connected helped me understand what each line was doing. I also tested small parts of the code, like checking if the file path worked before adding the guessing part. Breaking the problem into smaller sections and comparing it to past examples helped me figure out what to do next.

Learning

Was there anything you learned that you think will help you with future challenges?

Yes, I learned that using a try and catch statement for file reading will really help me with future challenges. It taught me how to safely open files, read information, and prevent the program from crashing if something goes wrong. I also learned that breaking a program into smaller parts and testing each one makes harder problems easier to solve. This will help me in future assignments because I can use the same step-by-step process when working with files or larger programs.